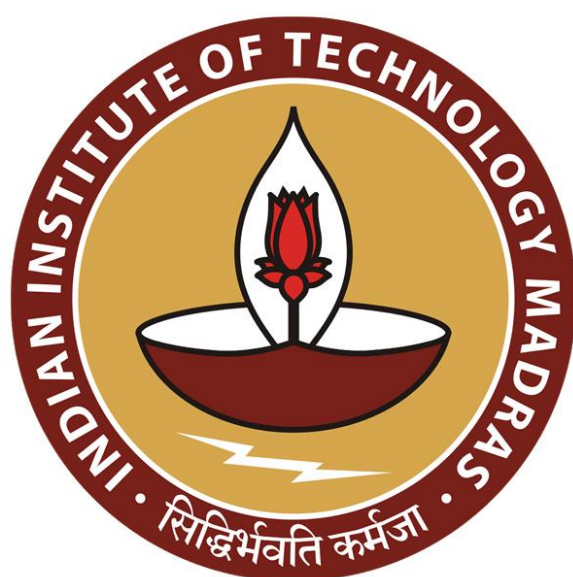




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Race Conditions

So, while introducing threads, we talked about race conditions.

(Refer Slide Time: 00:19)

Threads and shared variables

- Threads are lightweight processes with shared variables that can run in parallel
- Browser example: download thread and user-interface thread run in parallel
 - Shared boolean variable `terminate` indicates whether download should be interrupted
 - `terminate` is initially false
 - Clicking `Stop` sets it to true
 - Download thread checks the value of this variable periodically and aborts if it is set to true
- Watch out for **race conditions**
 - Shared variables must be updated consistently

Madhavan Mukund Race conditions Programming Concepts using Java 2/7

So, we said that threads are lightweight processes with shared variables. And we would like to use the shared variables to coordinate the thread. So, we took this example of this browser where we said that the download thread and the cancel thread would interact through a Boolean variable `terminate`. And by polling this variable, the download thread would decide whether to stop or not. So, this kind of concurrent access to a variable can create some kind of inconsistency. And this is what is called a race condition.

(Refer Slide Time: 00:53)



Maintaining data consistency

- `double accounts[100]` describes 100 bank accounts
- Two functions that operate on `accounts`: `transfer()` and `audit()`

```
boolean transfer (double amount,
                  int source,
                  int target){
    if (accounts[source] < amount){
        return false;
    }
    accounts[source] -= amount;
    accounts[target] += amount;
    return true;
}

double audit(){
    // total balance across all accounts
    double balance = 0.00;
    for (int i = 0; i < 100; i++){
        balance += accounts[i];
    }
    return balance;
}
```



Madhavan Mukund

Race conditions

Programming Concepts using Java 3/7

So, let us look at a more typical example of concurrent programming where race conditions are more meaningful. So, suppose we are maintaining some bank account information. So, as a simple situation, we have an array, in which we keep the balance in 100 accounts and the account numbers are implicitly 0 to 99. So, we have an account number, which is 0 to 99. And for each account, we have the current balance, and this is all stored in an array called `accounts`.

Now, we might want to allow two functions, which operate on this area of accounts. So, one function is a transfer function, it will take an amount, and it will take a source account, so some index between 0 and 99, and a target account some another account between 0 and 99. And transfer the amount from `i` to `j`, provided of course, that the account number `i` has enough balance. So, you want to take amount and transfer it from source to target. So, source and target should be within your array.

Otherwise, you will get some array bound exception, let us assume all that is taken care of. So, what we first check is whether the source account has enough money. If the amount that is stored currently in the balance of the source account is less than amount, then we just say that this transfer did not succeed. So, we have a Boolean, which just indicates whether the transfer worked or not. So, if you do not have enough balance, insufficient balance, you cannot transfer the money.

Otherwise, you do the obvious thing, it is you subtract the amount from the source, you add the amount of the target, and then you say the transfer is succeeded. And you return true. So, this is the transfer thread. The other thread is more kind of status check. So, this is something which I am calling audit. So, what it does is it runs through all 100 accounts, and adds up the balance. So, it just takes the sum of the balance from account 0 to account 99.

Now remember that in this, if I restrict my access to these two functions, this is not a realistic bank, what we are saying is we have a bank with 100 customers, and there is no external inflow or outflow, you cannot bring in money from outside the bank and deposit it. And you cannot take money from inside this bank and pay somebody, the only thing you can do with this bank is to move money from one account to another account.

So, therefore, it makes sense that if I know how much money was there in the bank across all these accounts, if I audit the bank once, then that value of the audit should remain unchanged. So, each time I call audit, I should get the same answer, I should get the sum of all the balances of all the accounts.

The individual balances may have changed because of transfers, but the total cannot change. If I give 10 rupees to you, then between us, we still have the same amount of money as before I gave 10 rupees to you. So, you have 10 rupees more, if 10 rupees less, that cancels out. So, that is exactly what we are assuming in this simple example.

(Refer Slide Time: 03:42)



Maintaining data consistency

- `double accounts[100]` describes 100 bank accounts
- Two functions that operate on `accounts`: `transfer()` and `audit()`
- What are the possibilities when we execute the following?

Thread 1

```
...  
status =  
transfer(500.00, 7, 8);  
...
```

Thread 2

```
...  
System.out.  
print(audit());  
...
```

```
boolean transfer (double amount,  
    int source,  
    int target){  
    if (accounts[source] < amount){  
        return false;  
    }  
    accounts[source] -= amount;  
    accounts[target] += amount;  
    return true;  
}  
  
double audit(){  
    // total balance across all accounts  
    double balance = 0.00;  
    for (int i = 0; i < 100; i++){  
        balance += accounts[i];  
    }  
    return balance;  
}
```

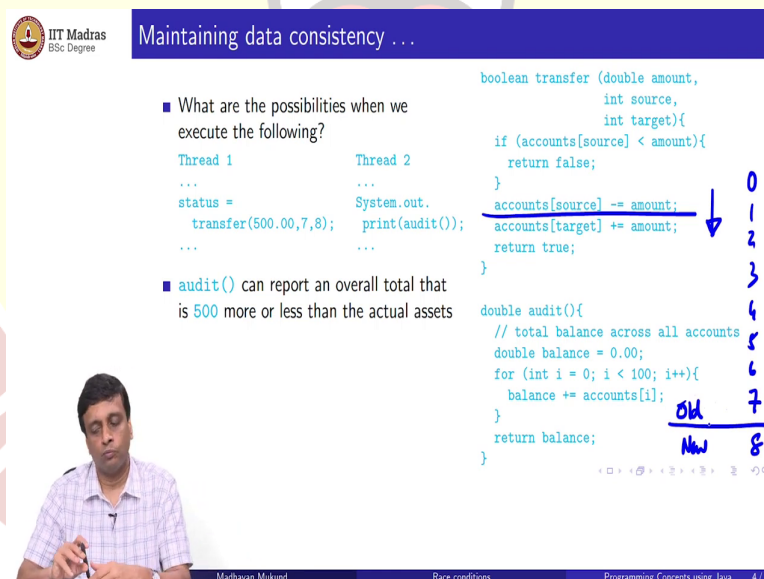


So, now we are talking about concurrency. So, the question that is obvious is what happens if I try to execute these two things in parallel. So, notice that one of them is updating accounts. So, accounts is an array. And one function, namely transfer is updating this by transferring money from one to another. So, it could reduce the entry in one position and increase the entry another position.

The other one is not changing anything, it is merely looking up the values of the accounts. So, remember what we said about the audit. In this particular example, because I cannot transfer money into or outside this array from anywhere else, the money inside the accounts across the accounts, 0 to 99 is fixed. So, the audit should in principle, always answer the same thing.

You should always give me the same value, which is the initial sum of the account 0 to account 99. So, here what I am doing is I am transferring say 500 rupees from account 7 to account 8. So, this is the change, I am making the accounts and in parallel now I am asking audit to report what is the situation?

(Refer Slide Time: 04:49)



Maintaining data consistency ...

- What are the possibilities when we execute the following?

Thread 1	Thread 2
...	...
status =	System.out.
transfer(500.00, 7, 8);	print(audit());
...	...

- audit() can report an overall total that is 500 more or less than the actual assets

```

boolean transfer (double amount,
                  int source,
                  int target){
    if (accounts[source] < amount){
        return false;
    }
    accounts[source] -= amount;
    accounts[target] += amount;
    return true;
}

double audit(){
    // total balance across all accounts
    double balance = 0.00;
    for (int i = 0; i < 100; i++){
        balance += accounts[i];
    }
    return balance;
}
    
```

Hand-drawn diagram of account array indices 0 to 99:

- Indices 0, 1, 2, 3, 4, 5, 6 are marked with a bracket and labeled 'Old'.
- Index 7 is marked with a bracket and labeled 'New'.
- Index 8 is marked with a bracket and labeled 'New'.

Madhavan Mukund Race conditions Programming Concepts using Java 4/7

So, I claim that the audit, of course, logically should give you the same answer, but it is not difficult to imagine why the audit can report 500 more or 500 less. So, why would it report 500 less? Well, it really depends on how this accounting is done. So, if I am counting these accounts, and audit is happening in parallel, so audit is reading 0 1 2 3 4 5 6. And then when it reads 7, it reads after minus 500. So, it is read.

So, in some sense, I am at this situation and my transfer. I am at this situation, my transfer, and I read this money before the money is transferred. Rather, after the money is removed from 7, but before it is transferred to it. So, as a result, the audit sees 500 less in the bank account 7. But at this point, because the transfer is not completed, when it goes on to 8, I get the old value. So, I have the value of account number 8, before the transfer has happened.

So, net, I have lost this 500 as far as the audit is concerned, because the 500 is gone from 7, so I did not count it there. But it has not yet reached 8, so I did not count it. So, it could be 500 less. How could it be 500 more? Well, supposing I did the following. I reach 7. So, I did it a different way. So, now the audit reached 7, and at this point, I got the old value of 7, so no money has left 7 yet. So, I got the old total. Then the audit for some reason stopped. So, remember that these are all happening in parallel.

And as I said, we are interleaving the operation. So, we cannot guarantee like we saw in that earlier thing where it was printing out the ID, each step of this might happen in a different order with respect to the other thing that is happening at that time. So, I stopped at 7. And now the audit has the old value of 7 before 500 so, whatever had happened before, then this happens in the transfer. So, now you see the new 8.

So, you see the 500 added to 8, although you did not see the 500 subtracted from 7. So, you have a total which is 500 more than you believe. So, in this situation, when I have these two parallel things, I could have, of course, the correct thing in case the interleaving of these things happened in a nice way. But if it happened in a not so nice way, I could either see 500 less or I could see 500 more.

(Refer Slide Time: 07:41)

Maintaining data consistency ...

- What are the possibilities when we execute the following?

```
Thread 1      Thread 2
...           ...
status =      System.out.
transfer(500.00,7,8); print(audit());
...           ...
```

- `audit()` can report an overall total that is 500 more or less than the actual assets
 - Depends on how actions of `transfer` are interleaved with actions of `audit`

```
boolean transfer (double amount,
                  int source,
                  int target){
    if (accounts[source] < amount){
        return false;
    }
    accounts[source] -= amount;
    accounts[target] += amount;
    return true;
}
```

```
double audit(){
    // total balance across all accounts
    double balance = 0.00;
    for (int i = 0; i < 100; i++){
        balance += accounts[i];
    }
    return balance;
}
```



Madhavan Mukund

Race conditions

Programming Concepts using Java

4/7

So, we are talking about how the actions are interleaved with audit. But the first example that we had said that if the transfer was interrupted in the middle, so I have transferred 700 out, I mean 500 out of accounts 7, then I audit 7, and then I audit 8, and then I transfer to 8. So, between transferring from 7 to 8, the audit finished both of them. So, in that case, this particular flow was executed, execution was interrupted in order to generate the minus 500 case.

(Refer Slide Time: 08:19)

Maintaining data consistency ...

- What are the possibilities when we execute the following?

```
Thread 1      Thread 2
...           ...
status =      System.out.
transfer(500.00,7,8); print(audit());
...           ...
```

- `audit()` can report an overall total that is 500 more or less than the actual assets
 - Depends on how actions of `transfer` are interleaved with actions of `audit`
 - Can even report an error if `transfer` happens *atomically*

```
boolean transfer (double amount,
                  int source,
                  int target){
    if (accounts[source] < amount){
        return false;
    }
    accounts[source] -= amount;
    accounts[target] += amount;
    return true;
}
```

```
double audit(){
    // total balance across all accounts
    double balance = 0.00;
    for (int i = 0; i < 100; i++){
        balance += accounts[i];
    }
    return balance;
}
```

7 before
& after



Madhavan Mukund

Race conditions

Programming Concepts using Java

4/7

But the plus 500 case was very different. In the plus 500 case, I could even assume that this whole thing happened without any interruption. It just is the case that this process 7 before and this process 8 after. So, it is not just a question of saying that the entire transfer must

happen at one time, even if the entire transfer happens at one time, because this is a kind of complex data type. So, the transfer is affecting some part of it.

And when I am processing this complex data type in a loop, I might see the data type in different states before and after that transfer. And those two states may be incompatible with each other. So, this is the kind of situation that we are really worried about. And this is what we want to deal with.

(Refer Slide Time: 09:09)

IIT Madras
BSc Degree

Atomicity of updates

- Two threads increment a shared variable `n`

Thread 1	Thread 2
...	...
<code>m = n;</code>	<code>k = n;</code>
<code>m++;</code>	<code>k++;</code>
<code>n = m;</code>	<code>n = k;</code>
...	...

Madhavan Mukund Race conditions Programming Concepts using Java 5/7

So that was a fairly complicated example using arrays. But let us look at a much simpler example. Supposing I just have one integer, I just have one integer which is shared by two threads. And this is a shared variable. And what I will assume is they also have local variables which are not shared.

So, each of them will copy the shared variable `n`, so `n` is a shared variable. So, each of them copies this `n` into its local variable `m` and `k` respectively. Then you increment the local variable, so you are not touching the last variable at all. And then you copy it back. So, if I start with an initial value of `n`, say 7, then one of them will increment it to 8 and the other one should increment it to 9.

(Refer Slide Time: 09:54)

Atomicity of updates

- Two threads increment a shared variable `n`

```
Thread 1      Thread 2
m = n;        k = n;
m++;          k++;
n = m;        n = k;
...          ...
```

- Expect `n` to increase by 2 ...
- ... but, time-slicing may order execution as follows

```
Thread 1: m = n;
Thread 1: m++;
Thread 2: k = n; // k gets the original value of n
Thread 2: k++;
Thread 1: n = m;
Thread 2: n = k; // Same value as that set by Thread 1
```



Madhavan Mukund

Race conditions

Programming Concepts using Java

5/7

So, logically, if this happens nicely, you would expect two increments to happen on it. So, it expected to increase by 2. But again because of interleaving, what could happen is that both of them end up seeing the same value of `n`. So, this one region and increments, but before it writes it back, the other one reads the old value in increments.

So, say the initial value of 17, then `m` is gone from 17 to 18, `k` is also gone from 17 to 18. So, now I will write back 18 from `m`, and then I will again write back 18 from `n`. So, I have lost this one update. So, I have made a kind of incorrect update, which is not logical in some sense because one increment has been lost.

(Refer Slide Time: 10:37)

Race conditions and mutual exclusion

- Race condition** — concurrent update of shared variables, unpredictable outcome
 - Executing `transfer()` and `audit()` concurrently can cause `audit()` to report more or less than the actual assets
- Avoid this by insisting that `transfer()` and `audit()` do not interleave
- Never simultaneously have current control point of one thread within `transfer()` and another thread within `audit()`

```
boolean transfer (double amount,
                  int source,
                  int target){
    if (accounts[source] < amount){
        return false;
    }
    accounts[source] -= amount;
    accounts[target] += amount;
    return true;
}

double audit(){
    // total balance across all accounts
    double balance = 0.00;
    for (int i = 0; i < 100; i++){
        balance += accounts[i];
    }
    return balance;
}
```



Madhavan Mukund

Race conditions

Programming Concepts using Java

6/7

So, this is technically what we call a race condition. So, a race condition is when we have concurrent updates. So, concurrent updates in the previous example, two things were updating. In the previous example, we had an update happening with a lookup. In the bank account case, the update was not concurrent, the transfer was happening sequentially first subtract then add. But it was happening in parallel with something that is observing it and concurrence.

In all such situations where I have this kind of concurrent update. And as a result of the concurrent update, there is some ambiguity about what the final answer is going to be. So, there is an unpredictable outcome. I call these race conditions and race conditions, clearly, I think that we should not see because there is always a possibility that we see the right outcome, it is not that we are guaranteed to make a mistake.

So, even in the transfer case, if I ran the audit, so ideally, for instance, if you ran it without doing it in parallel, you run the entire transfer, and then run the audit, or you first increment it in thread 1, and then you increment in thread 2, there is always sensible way to get the answer by just sequentially using it by running one thread at a time. But we cannot rule out the possibility that something has gone wrong, and that is what we want to guard against as a race condition.

So, as I said, if we insist that the entire thread must execute. So, earlier, we said that transfer may execute as an atomic unit, but if audit is happening in parallel, so that it has seen 0 to 6, or 0 to 7 before and then 8 onwards, after that atomic thing, it does not help. So, both must be atomic.

If I do all the transfer, before all the audit, or all the audit before all the transfer, nothing can go wrong. But, you do not want in some sense, both of these threads to be executing in the middle at the same time. So, this is kind of, so we do not want this to be somewhere here and this to be somewhere here. So, this is the kind of thing that we want to rule against.

(Refer Slide Time: 12:35)

- **Race condition** — concurrent update of shared variables, unpredictable outcome
 - Executing `transfer()` and `audit()` concurrently can cause `audit()` to report more or less than the actual assets
- Avoid this by insisting that `transfer()` and `audit()` do not interleave
- Never simultaneously have current control point of one thread within `transfer()` and another thread within `audit()`
- **Mutually exclusive access to critical regions of code**

```
boolean transfer (double amount,
                  int source,
                  int target){
    if (accounts[source] < amount){
        return false;
    }
    accounts[source] -= amount;
    accounts[target] += amount;
    return true;
}

double audit(){
    // total balance across all accounts
    double balance = 0.00;
    for (int i = 0; i < 100; i++){
        balance += accounts[i];
    }
    return balance;
}
```



Madhavan Mukund

Race conditions

Programming Concepts using Java 6/7

So, this section, which deals with accounts, and this section, which updates accounts, these are operating on this concurrent shared data, and these are what are called critical sections. If I am doing something here, and something else is changing it or if I am changing something here and something else is observing it, then there is a possibility of a race condition.

So, what we want to ensure is that these two so we are not so concerned about whether they are inside the code or not, but they should not be certainly inside this section at the same time. So, if one is updating the other should not be observing and vice versa. So, this is called a critical section or a critical region. And what we want to do somehow ensure that when we run these concurrent threads, we have a way of controlling what is called mutual exclusive access that is either one is there or the other is there, we do not want both of them to simultaneously be there.

(Refer Slide Time: 13:28)

- Concurrent update of a shared variable can lead to data inconsistency
 - **Race condition**
- Control behaviour of threads to regulate concurrent updates
 - **Critical sections** — sections of code where shared variables are updated
 - **Mutual exclusion** — at most one thread at a time can be in a critical section



Madhavan Mukund

Race conditions

Programming Concepts using Java 7/7

So, this is going to be now the focus of what we are going to be looking at in terms of concurrent programming. So, we have these race conditions which are caused by concurrent updates, and they can lead to this inconsistent view of the data. So, we want to look and identify how to control access to these critical sections of critical regions. So, that these sections of code where variables are updated, or checked.

Not just updated, but also examined are not simultaneously executed. And this is called mutual exclusion. We want to make sure that more than one thread cannot be in its critical region at the same time. So, all the critical regions across threads correspond to some shared data, and at most one of them can be either viewing it or updating it at one time.